

1. Quantum-centric representations (adjacent to QASM)

◇ OpenQASM

You already know this one.

- **Level:** Gate-level quantum assembly
 - **Strength:** Hardware-close, explicit operations
 - **Limitation:** No multi-level abstraction, weak for fault-tolerance reasoning beyond surface code assumptions
 - **Use case:** Final lowering, not analysis
-

◇ QIR (LLVM-based)

- Developed by Microsoft
- **LLVM IR + quantum intrinsics**
- Classical + quantum control flow in one IR
- Fault tolerance is *implicit* (assumes error-corrected backend)

Why it matters to you

- Shows how **domain-specific ops live inside LLVM**
 - Excellent example of “*extend LLVM rather than replace it*”
-

◇ Cirq internal IR

- DAG-based circuit IR
- Hardware-aware scheduling
- Strong on noise modeling

Relevance

- Graph-based IRs are excellent for **fault propagation analysis**
 - Weak as a general compiler IR
-

◇ t|ket> IR

- Graph + rewrite-rule-based IR
- Focus on circuit equivalence & optimization
- Hardware-specific rewrites

Lesson

- Rewrite systems are *very powerful* for fault-aware transformations
-

2. MLIR ecosystem (beyond “just MLIR”)

MLIR is not one IR — it's a **meta-IR framework**.

◇ **MLIR Dialects**

Some important ones:

- **LLVM Dialect** → lowers to LLVM IR
- **SCF / CF Dialects** → structured / control flow
- **Affine Dialect** → loop bounds, memory access (great for verification)
- **GPU / Vector / Linalg Dialects** → accelerator-friendly
- **Custom Dialects** → *this is where fault tolerance belongs*

Why MLIR is dominant for compiler chips

- Multiple abstraction layers
 - Verifiers at every level
 - Clean lowering semantics
 - Perfect for *fault-aware lowering pipelines*
-

3. Classical compiler IRs (still extremely relevant)

◇ **LLVM IR**

- SSA, typed, target-agnostic
- Massive tooling ecosystem

Fault-tolerance angle

- Instruction duplication
- Control-flow checking
- Memory hardening
- Sanitizer-style instrumentation

LLVM IR is where **fault tolerance becomes concrete code**.

◇ **GIMPLE**

- GCC's SSA-based IR
- Less extensible than LLVM
- Strong optimization theory

Mostly academic relevance today

◇ **Sea of Nodes**

- Used in HotSpot / Graal
- Program = graph of data + control dependencies

Why it matters

- Fault propagation becomes *graph reachability*
 - Excellent for speculative and redundant execution modeling
-

4. Hardware / accelerator-centric IRs (important for “compiler chips”)

◇ FIRRTL

- Used by Chisel
- Hardware-semantic IR
- Explicit wires, registers, clocks

Fault tolerance

- ECC modeling
- Redundant pipelines
- Lockstep cores

If your “compiler chip” includes **co-designed hardware**, FIRRTL-like IRs are critical.

◇ RTL

- Verilog/VHDL abstraction
 - Not a compiler IR, but relevant downstream
-

5. Dataflow & graph IRs (excellent for fault modeling)

◇ Dataflow IR

- Nodes = ops, edges = values
- No implicit control flow

Why fault tolerance loves dataflow

- Easy redundancy
- Natural voting
- No hidden state

Common in:

- DSP
 - Streaming accelerators
 - AI chips
-

◇ DFG

- Used inside many compilers
 - Often derived from SSA
-

6. Formally verified IRs (for *strong* fault guarantees)

◊ **CompCert IRs**

- Multiple formally defined IR levels
- Machine-checked correctness proofs

Relevance

- If your fault tolerance has **safety-critical requirements**, this philosophy matters
 - Heavyweight, but inspiring
-

◊ **WhyML**

- IR for program proofs
 - Not for codegen, but for *guarantees*
-

7. Runtime / VM-level IRs (underrated)

◊ **WebAssembly**

- Portable, sandboxed, deterministic
- Explicit control and memory

Fault tolerance angle

- Strong isolation
 - Deterministic replay
 - Easy duplication at runtime
-

◊ **JVM bytecode**

- Stack-based
 - Mature verification
 - Runtime fault handling
-

8. How these compare (quick mental model)

IR Type	Best for	Fault-tolerance leverage
QASM	Final quantum execution	Low
QIR	Hybrid classical/quantum	Medium
LLVM IR	Concrete code hardening	High
MLIR	Structured fault-aware compilation	Very High
Dataflow IR	Fault propagation & redundancy	Very High

IR Type	Best for	Fault-tolerance leverage
FIRRTL	Hardware-level resilience	High
Verified IRs	Formal guarantees	Very High (but heavy)
WASM	Isolation & sandboxing	Medium

9. The **correct mental model** (important)

Think in **layers**, not “pick one IR”:

**High-level semantic IR → Fault-annotated IR → Redundancy-expanded IR → Hardware-aware IR
→ ISA / microcode**

That’s exactly why **MLIR + domain dialects + LLVM/QASM backends** is emerging as the dominant pattern.